

实验一、线性表的应用

班级：数据 231

学号：235127

姓名：艾洋

[1] 实验内容：

1.约瑟夫环

N 个人围成一圈顺序编号，从 1 号开始按 1、2、3.....顺序报数，报 p 者退出圈外，其余的人再从 1、2、3 开始报数，报 p 的人再退出圈外，以此类推。请按退出顺序输出每个退出人的原序号。

输入格式：

输入只有一行，包括一个整数 N($1 \leq N \leq 3000$)及一个整数 p($1 \leq p \leq 5000$)。

输出格式：

按退出顺序输出每个退出人的原序号，数据间以一个空格分隔，但行尾无空格。

输入样例：

在这里给出一组输入。例如：

7 3

输出样例：

3 6 2 7 5 1 4

代码长度限制 16 KB

时间限制 400 ms

内存限制 64 MB

2.重排链表

给定一个单链表 $L_1 \rightarrow L_2 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$ ，请编写程序将链表重新排列为 $L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow \dots$ 。例如：给定 L 为 $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$ ，则输出应该为 $6 \rightarrow 1 \rightarrow 5 \rightarrow 2 \rightarrow 4 \rightarrow 3$ 。

输入格式：

每个输入包含 1 个测试用例。每个测试用例第 1 行给出第 1 个结点的地址和结点总个数，即正整数 N (≤ 105)。结点的地址是 5 位非负整数，NULL 地址用 -1 表示。

接下来有 N 行，每行格式为：

Address Data Next

其中 Address 是结点地址；Data 是该结点保存的数据，为不超过 10^5 的正整数；Next 是下一结点的地址。题目保证给出的链表上至少有两个结点。

输出格式：

对每个测试用例，顺序输出重排后的结果链表，其上每个结点占一行，格式与输入相同。

输入样例：

00100 6

00000 4 99999

00100 1 12309

68237 6 -1

33218 3 00000

99999 5 68237

12309 2 33218

输出样例：

68237 6 00100

00100 1 99999

99999 5 12309

12309 2 00000

00000 4 33218

33218 3 -1

代码长度限制 16 KB

时间限制 500 ms

内存限制 64 MB

[2] 实验目的

掌握链表的基本操作：插入、删除、查找等运算，能够灵活应用链表这种数

据结构。

[3] 程序清单

第一题

```
#include<iostream>

using namespace std;

typedef struct node
{
    int data;
    struct node* next;
}Node;

void ysf_function(int N,int M)
{
    Node *head = NULL,*p=NULL,*r=NULL;
    head = (Node*)malloc(sizeof(Node));
    head->data=1;
    head->next=NULL;
    p=head;

    for(int i=2;i<=N;i++){
        r=(Node*)malloc(sizeof(Node));
        r->data=i;
        r->next=NULL;
        p->next=r;
        p=r;
    }

    p->next=head;
    p=head;
```

```

while(p->next!= p){
    for(int i=1;i<M;i++){
        r=p;
        p=p->next;
    }
    cout<<p->data<<" ";
    r->next=p->next;
    p=p->next;
}
cout<<p->data;
}

int main()
{
    int n,p;
    cin>>n>>p;
    ysf_function(n,p);
    return 0;
}

```

第二题

```

#include <iostream>
#include <iomanip>
using namespace std;

#define MAX_SIZE 100000

struct Node {
    int address;
    int data;
    int next;
}

```

```

};

int main() {
    int firstAddress, n;
    cin >> firstAddress >> n;

    Node nodes[MAX_SIZE];

    for (int i = 0; i < n; ++i) {
        int address, data, next;
        cin >> address >> data >> next;
        nodes[address] = {address, data, next};
    }

    // 使用数组保存链表节点
    Node linkedList[MAX_SIZE];
    int currentAddress = firstAddress;
    int listSize = 0;

    // 根据给定的地址顺序，将链表节点存储到 linkedList 数组中
    while (currentAddress != -1) {
        linkedList[listSize++] = nodes[currentAddress];
        currentAddress = nodes[currentAddress].next;
    }

    // 重新排列链表
    Node reorderedList[MAX_SIZE];
    int left = 0, right = listSize - 1, reorderedSize = 0;

    // 双指针交替取头尾节点

```

```

while (left <= right) {
    if (left == right) {
        reorderedList[reorderedSize++] = linkedList[left];
        break;
    } else {
        reorderedList[reorderedSize++] = linkedList[right];
        reorderedList[reorderedSize++] = linkedList[left];
        ++left;
        --right;
    }
}

for (int i = 0; i < reorderedSize; ++i) {
    int address = reorderedList[i].address;
    int data = reorderedList[i].data;
    int next = (i + 1 < reorderedSize) ? reorderedList[i + 1].address : -1;
    printf("%05d %d ", address, data);

    // 输出下一个地址，如果是最后一个节点，则 next 输出 -1
    if (next == -1)
        printf("-1\n");
    else
        printf("%05d\n", next);
}

return 0;

```

[4]调试步骤

问题一：

1.在构建链表时，代码通过循环逐个连接每个节点。链表的结构在循环结束后应该是一个闭环，即最后一个节点的 `next` 指向 `head`。

调试步骤：确认所有节点都正确地连接，并且最后一个节点的 `next` 指向 `head`。

2.约瑟夫环问题的核心是每次从当前节点开始计数，删除第 `M` 个节点，直到只剩下一个节点。

调试步骤：在 `while (p->next != p)` 循环中，每次删除节点后，应该检查当前节点的删除操作是否符合约瑟夫环的规则。注意 `while (p->next != p)` 是退出循环的条件，表示只剩一个节点时退出。

问题二：

1.在代码中，通过 `currentAddress` 跟踪当前节点的地址，并将节点按顺序存储在 `linkedList` 数组中，直到到达链表的尾部（`next == -1`）。确保节点的链表顺序被正确地解析并存储到 `linkedList` 中。

调试步骤：打印链表构建过程，输出存储到 `linkedList` 中的每个节点的 `address` 和 `next`，以确保节点按照链表顺序正确存储。

2.重新排列链表时，使用了双指针技术（`left` 和 `right`）从头尾交替选取节点。这个过程的关键是确保 `left` 指针和 `right` 指针在正确的范围内移动，并且不会越界。

调试步骤：打印每次选择的节点，检查是否按头尾交替的顺序正确处理了节点。

[5] 运行结果:

问题一:

```
Microsoft Visual Studio 调试
7 3
3 6 2 7 5 1 4
D:\autumn\代码\Exercise 24 summer\数据结构(zju)\x64\Debug\数据结构(zju).exe (进程 9932)已退出, 代码为 0。
按任意键关闭此窗口...
```

提交结果			×
题目	用户	提交时间	
7-1	235127 艾洋	2024/11/15 17:05:41	
编译器	内存	用时	
C++ (g++)	524 / 65536 KB	45 / 400 ms	
状态 ?	评测时间		
答案正确	2024/11/15 17:05:42		

问题二:

C++ (g++) | ⌵ ⌚ ⚙️ ?

测试用例 | 运行结果 | 编译器输出

复制内容

1

68237.6.00100

2

00100.1.99999

3

99999.5.12309

4

12309.2.00000

5

00000.4.33218

6

33218.3.-1

7



提交结果			×
题目	用户	提交时间	
7-2	235127 艾洋	2024/11/15 17:07:16	
编译器	内存	用时	
C++ (g++)	5712 / 65536 KB	90 / 500 ms	
状态 ?	评测时间		
答案正确	2024/11/15 17:07:16		

[6] 分析与思考

通过这次约瑟夫环实验，我加深了对循环链表的理解，特别是在处理循环结构和节点删除时的应用。循环链表的闭环特性使得链表操作更加简便，特别适合解决像约瑟夫环这样需要不断循环遍历节点的问题。通过逐个删除节点并调整指针，能够模拟报数的过程，直到最终剩下一个节点。在实验过程中，我认识到指针操作对于链表的正确性至关重要，任何细微的错误都可能导致链表断裂或无限循环，这使得我对链表操作的细节有了更深刻的体会。

另外，实验也让我意识到链表的内存管理尤为重要，尤其是动态内存分配时，必须保证每个节点被正确释放，以避免内存泄漏。尽管本次实验中处理较为简单，但在面对更大规模的数据时，如何优化链表结构和操作效率将成为一个重要的课题。总的来说，循环链表为解决约瑟夫环问题提供了直观且高效的方式，而通过这次实验，我不仅掌握了循环链表的基本操作，也积累了处理更复杂链表问题的经验。

实验二、栈与队列的应用

河北工业大学课程共享计划

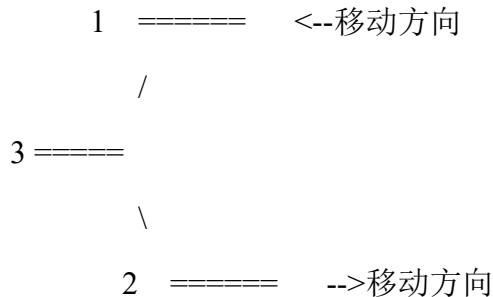
班级：数据 231

学号：235127

姓名：艾洋

[1] 实验内容:

1.列车厢调度



大家或许在某些数据结构教材上见到过“列车厢调度问题”（当然没见过也不要紧）。今天，我们就来实际操作一下列车厢的调度。对照上方的 ASCII 字符图，问题描述如下：

有三条平行的列车轨道（1、2、3）以及 1-3 和 2-3 两段连接轨道。现有一列车厢停在 1 号轨道上，请利用两条连接轨道以及 3 号轨道，将车厢按照要求的顺序转移到 2 号轨道。规则是：

每次转移 1 节车厢；

处在 1 号轨道的车厢要么经过 1-3 连接道进入 3 号轨道（该操作记为"1->3"），要么经过两条连接轨道直接进入 2 号轨道（该操作记为"1->2"）；

一旦车厢进入 2 号轨道，就不要再移出该轨道；

处在 3 号轨道的车厢，只能经过 2-3 连接道进入 2 号轨道（该操作记为"3->2"）；

显然，任何车厢不能穿过、跨越或绕过其它车厢进行移动。

对于给定的 1 号停车顺序，如果经过调度能够实现 2 号轨道要求的顺序，则给出操作序列；如果不能，就反问用户 Are(你) you(是) kidding(凯丁) me(么)？

输入格式：

两行由大写字母组成的非空字符串，第一行表示停在 1 号轨道上的车厢从左到右的顺序，第二行表示要求车厢停到 2 号轨道的进道顺序（输入样例 1 中第二行 CBA 表示车厢在 2 号轨道的停放从左到右是 ABC，因为 C 最先进入，所以在最右边）。两行字符串长度相同且不超过 26（因为只有 26 个大写字母），每个字母表示一节车厢。题目保证同一行内的字母不重复且两行的字母集相同。

输出格式：

如果能够成功调度，给出最短的操作序列，每个操作占一行。所谓“最短”，

即如果 1->2 可以完成的调度，就不要通过 1->3 和 3->2 来实现。如果不能调度，输出 "Are you kidding me?"

输入样例 1:

ABC

CBA

输出样例 1:

1->3

1->3

1->2

3->2

3->2

输入样例 2:

ABC

CAB

输出样例 2:

Are you kidding me?

代码长度限制

16 KB

时间限制

400 ms

内存限制

64 MB

栈限制

8192 KB

2.银行业务队列简单模拟

设某银行有 A、B 两个业务窗口，且处理业务的速度不一样，其中 A 窗口处理速度是 B 窗口的 2 倍 —— 即当 A 窗口每处理完 2 个顾客时，B 窗口处理

完 1 个顾客。给定到达银行的顾客序列，请按业务完成的顺序输出顾客序列。假定不考虑顾客先后到达的时间间隔，并且当不同窗口同时处理完 2 个顾客时，A 窗口顾客优先输出。

输入格式:

输入为一行正整数，其中第 1 个数字 $N(\leq 1000)$ 为顾客总数，后面跟着 N 位顾客的编号。编号为奇数的顾客需要到 A 窗口办理业务，为偶数的顾客则去 B 窗口。数字间以空格分隔。

输出格式:

按业务处理完成的顺序输出顾客的编号。数字间以空格分隔，但最后一个编号后不能有多余的空格。

输入样例:

8 2 1 3 9 4 11 13 15

输出样例:

1 3 2 9 11 4 13 15

代码长度限制

16 KB

时间限制

400 ms

内存限制

64 MB

栈限制

8192 KB

[2] 实验目的

深入理解这两种线性数据结构的基本概念和工作原理，包括栈的后进先出（LIFO）和队列的先进先出（FIFO）特性。通过亲手实现和操作这些数据结构，学生将学会如何使用数组或链表等底层数据结构来构建栈和队列，并掌握它们的基本操作，如入栈、出栈、入队、出队等。

[3] 程序清单

7-1 列车厢调度

```
#include<iostream>
#include<stack>
#include<cstring>
using namespace std;
# define N 100
//理解：3 作为栈,1 元素符合 2 就移动，不符合进栈，但每动一次还要检测栈顶元素和目标的一致性
```

```
int main(){
    char initial[N],last[N];cin>>initial>>last;
    stack<char> track3;
    int target = 0;//3 的目标索引
    int len = strlen(initial);
    char operations[N][5];//记录操作
    int o = 0;//记录操作次数（也是下标）

    for(int i=0;i<len;i++){
        if(last[target]==initial[i]){
            sprintf(operations[o], "1->2");
            o++;
            target++;
        }
        else{
            track3.push(initial[i]);
            sprintf(operations[o], "1->3");
            o++;
        }
        while (!track3.empty() && track3.top() ==last[target]) {
            track3.pop();
            sprintf(operations[o], "3->2");
            o++;
            target++;
        }
    }

    while (!track3.empty() && track3.top() == last[target]) {//1 空了之后 3 可能还有元素
        track3.pop();
        sprintf(operations[o], "3->2");
        o++;
        target++;
    }
}
```

```

    }

    if(target==len){//检查是否全部完成
        for (int i=0;i<o;i++)    cout<<operations[i]<<endl;
    }
    else cout<<"Are you kidding me?\n"<<endl;
    return 0;
}

```

7-2 银行业务队列简单模拟

```

#include<iostream>
#include <queue>
#include <vector>
using namespace std;

int main(){
    int n;cin>>n;
    vector<int> people(n);
    vector<int> result;
    queue<int> qA,qB;
    for(int i=0;i<n;i++){
        cin>>people[i];
        if(people[i]%2==1)    qA.push(people[i]);
        else qB.push(people[i]);
    }

    while(!qA.empty() || !qB.empty()){
        //先处理 A
        if(qA.size()>=2){
            result.push_back(qA.front());
            qA.pop();
            result.push_back(qA.front());
            qA.pop();
        }
        else if (!qA.empty()){//这里要写上条件！ 要不然空了还在访问
            result.push_back(qA.front());
            qA.pop();
        }

        //处理 B
        if(!qB.empty()){
            result.push_back(qB.front());
            qB.pop();
        }
    }
}

```



```
    }

    for(int i=0;i<n;i++){
        if(i!=n-1)    cout<<result[i]<<" ";
        else    cout<<result[i];
    }
    return 0;
}
```

[4]调试步骤

问题一：

1.验证基本功能

简单案例验证：用一个非常简单的案例（如车厢只有一个，或者车厢的顺序完全相同）进行测试，看看程序是否能正常处理这些简单情况。

检查基础操作：验证程序是否能够正确执行单步操作（如 1->3, 1->2, 3->2），确认每一步操作后轨道的状态是否符合预期。

2.检查状态变化与边界条件

状态变更：逐步检查每个操作后，程序的轨道状态（track1, track2, track3）是否按预期发生变化。

边界条件：测试一些边界情况，比如：

track1 和 track3 为空时的行为。

track2 已经是目标顺序时的处理。

只有一列车厢的情况。

问题二：

1.逐步模拟并输出结果

循环处理：每次从 A 窗口队列和 B 窗口队列中取出顾客并输出。

A 窗口每次处理 2 个顾客，检查队列是否有足够的顾客。如果有，处理两个顾客

并输出。

B 窗口每次处理 1 个顾客，检查队列是否有顾客。如果有，处理一个顾客并输出。

优先处理 A 窗口顾客：当 A 和 B 窗口都能同时处理顾客时，优先输出 A 窗口顾客。

结束条件：当两个队列都为空时，停止输出。

2. 调试关键点

边界情况：顾客总数很少（如 $N=1$ ），或者所有顾客都被分配到一个窗口时，检查程序能否正确处理这些情况。

循环终止：确保循环正确终止。当 A 窗口和 B 窗口都没有顾客可以处理时，程序应停止输出。

队列的顺序：检查顾客按正确的顺序输出，A 窗口优先处理并输出其顾客，B 窗口处理其顾客。

[5] 运行结果:

问题一：

测试用例

运行结果

编译器输出

复制内容

1

1->3

2

1->3

3

1->2

4

3->2

5

3->2

6

问题二：

```

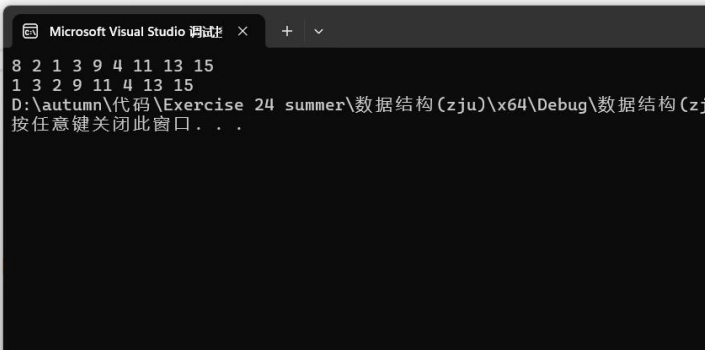
    result.push_back(qA.front());
    qA.pop();
}
else if (!qA.empty()) { //这里要写上条件! 要不然空了还在访问
    result.push_back(qA.front());
    qA.pop();
}

//处理B
if (!qB.empty()) {
    result.push_back(qB.front());
    qB.pop();
}

for (int i = 0; i < n; i++) {
    if (i != n - 1)    cout << result[i] << " ";
    else               cout << result[i];
}

return 0;
}

```



[6] 分析与思考

在解决“列车厢调度”这个问题时，我深刻体会到了栈在模拟实际调度问题中的强大作用。通过将 1 号和 3 号轨道视为两个栈，我学会了如何运用栈的后进先出特性来安排车厢的移动。这个问题不仅考验了我对栈操作的熟练度，还锻炼了我的逻辑思维和问题解决能力，特别是在处理顺序和调度方面。通过这个实验，我更加明白了栈在实际应用中的价值，以及如何灵活运用栈来解决复杂问题。

而在处理“银行业务队列简单模拟”这个问题时，我深入理解了队列在模拟排队系统中的作用，以及如何根据业务规则来调整顾客的处理顺序。我学会了如何使用队列来模拟顾客的等待和处理过程，并根据窗口处理速度的不同来优化顾客的服务顺序。这个实验让我认识到了队列在现实世界中的重要性，以及如何设计算法来提高服务效率和顾客满意度。通过这些实践，我不仅提升了我的编程技能，还增强了我对数据结构在解决实际问题中的应用能力。

实验三、树的应用

河北工业大学课程共享计划

班级：数据 231

学号：235127

姓名：艾洋

[1] 实验内容:

7-1 哈夫曼编码

给定一段文字，如果我们统计出字母出现的频率，是可以根据哈夫曼算法给出一套编码，使得用此编码压缩原文可以得到最短的编码总长。然而哈夫曼编码并不是唯一的。例如对字符串"aaaxuaxz"，容易得到字母 'a'、'x'、'u'、'z' 的出现频率对应为 4、2、1、1。我们可以设计编码 {'a'=0, 'x'=10, 'u'=110, 'z'=111}，也可以用另一套 {'a'=1, 'x'=01, 'u'=001, 'z'=000}，还可以用 {'a'=0, 'x'=11, 'u'=100, 'z'=101}，三套编码都可以把原文压缩到 14 个字节。但是 {'a'=0, 'x'=01, 'u'=011, 'z'=001} 就不是哈夫曼编码，因为用这套编码压缩得到 00001011001001 后，解码的结果不唯一，"aaaxuaxz" 和 "aazuaxax" 都可以对应解码的结果。本题就请你判断任一套编码是否哈夫曼编码。

输入格式：

首先第一行给出一个正整数 N ($2 \leq N \leq 63$)，随后第二行给出 N 个不重复的字符及其出现频率，格式如下：

$c[1] \ f[1] \ c[2] \ f[2] \ \dots \ c[N] \ f[N]$

其中 $c[i]$ 是集合 {'0' - '9', 'a' - 'z', 'A' - 'Z', '_'} 中的字符； $f[i]$ 是 $c[i]$ 的出现频率，为不超过 1000 的整数。再下一行给出一个正整数 M (≤ 1000)，随后是 M 套待检的编码。每套编码占 N 行，格式为：

$c[i] \ code[i]$

其中 $c[i]$ 是第 i 个字符； $code[i]$ 是不超过 63 个 '0' 和 '1' 的非空字符串。

输出格式：

对每套待检编码，如果是正确的哈夫曼编码，就在一行中输出 "Yes"，否则输出 "No"。

注意：最优编码并不一定通过哈夫曼算法得到。任何能压缩到最优长度的前缀编码都应被判为正确。

输入样例：

7

A 1 B 1 C 1 D 3 E 3 F 6 G 6

4

A 00000

B 00001

C 0001

D 001

E 01

F 10

G 11

A 01010

B 01011

C 0100

D 011

E 10

F 11

G 00

A 000

B 001

C 010

D 011

E 100

F 101

G 110

A 00000

B 00001

C 0001

D 001

E 00

F 10

G 11

输出样例：

Yes

Yes

No

No

代码长度限制

16 KB

时间限制

400 ms

内存限制

64 MB

栈限制

8192 KB

7-2 线索二叉树的建立和遍历

分数 300

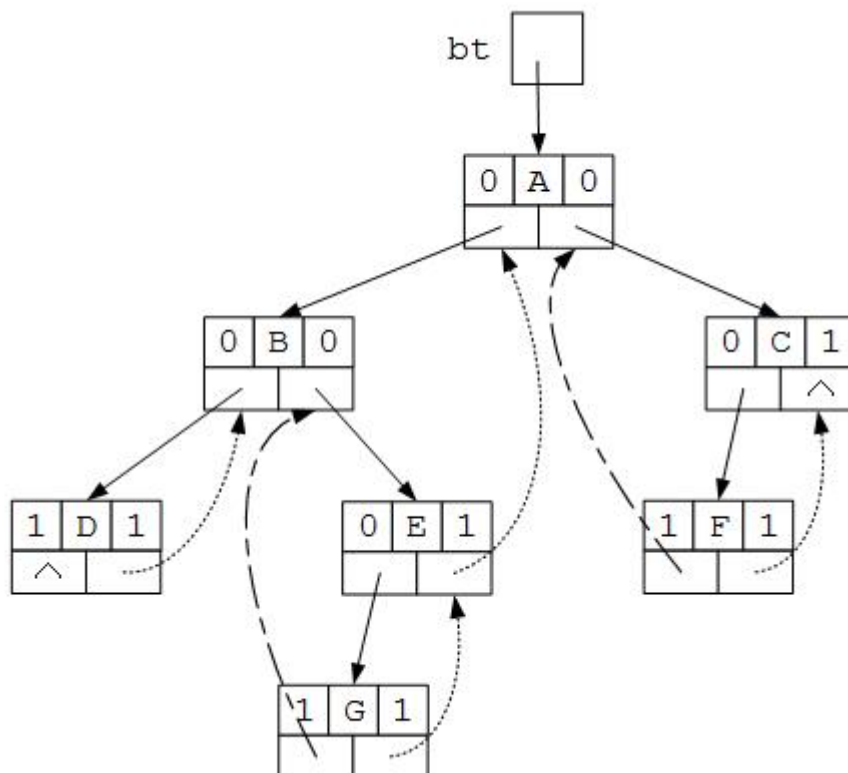
全屏浏览

切换布局

作者 张瑞霞

单位 桂林电子科技大学

本题要求实现对建立中序线索二叉树和中序遍历中序线索二叉树。



输入格式:

输入为先序序列

输出格式:

输出为中序遍历线索树的结点值以及结点的左右指针信息。

输入样例:

在这里给出一组输入。例如:

AB@D@@CE@@@

输出样例:

B 1 0

D 1 1

A 0 0

E 1 1

C 0 1

代码长度限制

16 KB

时间限制

400 ms

内存限制

64 MB

栈限制

8192 KB

[2] 实验目的

帮助学生深入理解树的基本概念和术语，掌握树的表示方法，以及实现树的基本操作，如节点的插入、删除和查找。通过实验，学生将学习树的遍历算法，包括前序、中序、后序和层序遍历，并能够将树结构应用于解决实际问题。

[3] 程序清单

实验一：

```
#include<iostream>
#include<vector>
#include<cstring>
#include <algorithm>
using namespace std;
#define N 1000
```

//只要保证任意一个编码不是另一个字符的前缀码即可，不用每次分别构造再比较

```
void swap(char* a, char* b) {
    char temp[N];
    strcpy(temp, a);
    strcpy(a, b);
    strcpy(b, temp);
}
```

```
int main() {
    int n;cin>>n;
    char c[N];
    int f[N];
    for(int i=0;i<n;i++){
        cin>>c[i]>>f[i];
    }
    int m;cin>>m;
```

```

char code[N][N];
while(m--) { // b 本来用的 for，但是太多下标分辨不清楚
    for(int i=0; i<n; i++) {
        cin>>c[i]>>code[i]; // 输入的 c[] 不重复吗
    } // 输入完成，下面该比较了

    int flag = 0; // 标记是否存在前缀关系
    // 按编码长度排序（确保较短的编码在前）
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (strlen(code[j]) > strlen(code[j + 1])) {
                // 交换编码
                swap(code[j], code[j + 1]);
            }
        }
    }

    for(int i=0; i<n; i++) {
        for(int j=i+1; j<n; j++) {
            if(strncmp(code[j], code[i], strlen(code[i])) ==
0) { // strcmp 来比较编码的前缀关系
                flag=1;
                break;
            }
            else if(f[i]<f[j] &&
strlen(code[i])>strlen(code[j])) {
                flag=1;
                break;
            }
        }
    }

    if(flag==0)    cout<<"Yes"<<endl;
    else    cout<<"No"<<endl;
}
return 0;
}

```

实验二：

```

#include <iostream>
using namespace std;

```

```

struct Node {
    char data;
}

```

```

    Node* lchild, *rchild;
    int ltag, rtag;
    Node(char val) : data(val), lchild(nullptr), rchild(nullptr),
ltag(0), rtag(0) {}
};

Node* PreOrder() { //先序序列构建二叉树（可以知道每个节点情况，所以直接构建）
    char c; cin >> c;
    if (c == '@') return nullptr;
    else {
        Node* T = new Node(c);
        T->lchild = PreOrder();
        T->rchild = PreOrder();
        return T;
    }
}

Node* pre = nullptr; //存上一个访问的节点
void InThread(Node* root) { //建立中序线索二叉树
    if (root) {
        InThread(root->lchild);
        //设置 tag 域
        if (root->lchild == nullptr) root->ltag = 1;
        else root->ltag = 0;
        if (root->rchild == nullptr) root->rtag = 1;
        else root->rtag = 0;
        //连接前驱节点
        if (pre != nullptr) {
            if (pre->rtag == 1) pre->rchild = root;
            if (root->ltag == 1) root->lchild = pre;
        }
        pre = root;
        InThread(root->rchild);
    }
}

void InOrderThread(Node* root) { //中序遍历中序线索二叉树
    Node* p = root;
    while (p->ltag != 1) p = p->lchild;
    while (p) {
        cout << p->data << " " << p->ltag << " " << p->rtag << endl;
        if (p->rtag == 1) p = p->rchild;
        else {

```

```

        p=p->rchild;
        while(p->ltag != 1)
            p=p->lchild;
    }
}

int main() {
    Node* Tree=PreOrder();
    InThread(Tree);
    InOrderThread(Tree);
    return 0;
}

```

[4]调试步骤

实验一：

验证编码：

实现一个函数来验证给定的编码是否是有效的前缀编码。

检查每个编码是否唯一，并且没有字符编码是其他字符编码的前缀。

测试单个编码：

对于每一套待检测的编码，使用验证函数来检查它是否是有效的哈夫曼编码。

输出"Yes"或"No"。

实验二：

构建普通二叉树：

实现一个函数来根据给定的先序序列构建一个普通的二叉树。

使用递归或非递归方法都可以，但要确保树的结构正确。

转换为中序线索二叉树：

实现一个函数来将普通二叉树转换为中序线索二叉树。

确保每个节点的左、右线索被正确设置，以便能够追踪中序遍历的路径。

中序遍历中序线索二叉树：

实现一个函数来进行中序遍历中序线索二叉树。

使用非递归方法，利用线索来遍历。

[5] 运行结果:

测试用例

运行结果

编译器输出

运行结果

预期结果

复制内容

1 Yes
2 Yes
3 No
4 No
5



复制内容

1 Yes
2 Yes
3 No
4 No
5

测试用例		运行结果	编译器输出
		运行结果	预期结果
复制内容		🔒	复制内容
1	B · 1 · 0		1 B · 1 · 0
2	D · 1 · 1		2 D · 1 · 1
3	A · 0 · 0		3 A · 0 · 0
4	E · 1 · 1		4 E · 1 · 1
5	C · 0 · 1		5 C · 0 · 1
6			6

[6] 分析与思考

对于哈夫曼编码和线索二叉树的建立与遍历这两个实验，我们深刻体会到了树结构在数据压缩和树遍历优化中的核心作用。哈夫曼编码实验让我们深入理解了如何利用树结构的频率特性来构建最优前缀编码，这是一种将树的统计信息转化为数据压缩的有效方法。通过构建哈夫曼树并从中生成编码，我们不仅学习了树的构建和遍历算法，还掌握了如何通过树结构实现数据的有效压缩。线索二叉树实验则进一步强化了我们对树遍历算法的理解，特别是如何通过线索化技术将普通二叉树转换为方便非递归遍历的线索二叉树，这对于提高树遍历的效率和灵活性至关重要。

这两个实验不仅提升了我们对树结构理论知识的掌握，还锻炼了我们将这些理论应用于实际编程问题的能力。在哈夫曼编码中，我们学习了如何根据字符频率构建最优前缀编码树，并从中派生出编码方案。在线索二叉树的实验中，我们掌握了如何将树结构转换为线索二叉树，并实现了高效的中序遍历。这些技能对于理解和实现复杂的数据结构和算法至关重要，它们不仅增强了对树结构操作的熟练度，也为解决实际问题提供了有力的工具。通过这些实验，我们更加深刻地认识到了树结构在计算机科学中的广泛应用和重要性。

实验四、图的应用

河北工业大学课程共享计划

班级：数据 231

学号：235127

姓名：艾洋

[1] 实验内容：

7-1 六度空间

分数 300

全屏浏览

切换布局

作者 DS 课程组

单位 浙江大学

“六度空间”理论又称作“六度分隔（Six Degrees of Separation）”理论。这个理论可以通俗地阐述为：“你和任何一个陌生人之间所间隔的人不会超过六个，也就是说，最多通过五个人你就能够认识任何一个陌生人。”如图 1 所示。

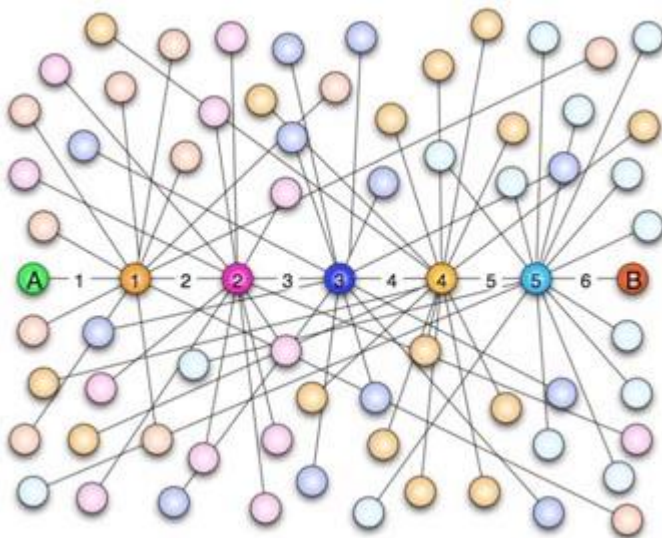


图 1 六度空间示意图

“六度空间”理论虽然得到广泛的认同，并且正在得到越来越多的应用。但是数十年来，试图验证这个理论始终是很多社会学家努力追求的目标。然而由于历史的原因，这样的研究具有太大的局限性和困难。随着当代人的联络主要依赖于电话、短信、微信以及因特网上即时通信等工具，能够体现社交网络关系的一手数据已经逐渐使得“六度空间”理论的验证成为可能。

假如给你一个社交网络图，请你对每个节点计算符合“六度空间”理论的结点占结点总数的百分比。

输入格式：

输入第 1 行给出两个正整数，分别表示社交网络图的结点数 N ($1 < N \leq 103$ ，表示人数)、边数 M ($\leq 33 \times N$ ，表示社交关系数)。随后的 M 行对应 M 条边，每行给出一对正整数，分别是该条边直接连通的两个结点的编号（节点从 1 到 N 编号）。

输出格式：

对每个结点输出与该结点距离不超过 6 的结点数占结点总数的百分比，精确到小数点后 2 位。每个结节点输出一行，格式为“结点编号: (空格) 百分比%”。

输入样例：

10 9

1 2

2 3
3 4
4 5
5 6
6 7
7 8
8 9
9 10

输出样例:

1: 70.00%
2: 80.00%
3: 90.00%
4: 100.00%
5: 100.00%
6: 100.00%
7: 100.00%
8: 90.00%
9: 80.00%
10: 70.00%

代码长度限制

16 KB

时间限制

2500 ms

内存限制

64 MB

栈限制

8192 KB

7-2 家庭房产

分数 300

全屏浏览

切换布局

作者 陈越

单位 浙江大学

给定每个人的家庭成员和其自己名下的房产，请你统计出每个家庭的人口数、人均房产面积及房产套数。

输入格式：

输入第一行给出一个正整数 N (≤ 1000)，随后 N 行，每行按下列格式给出一个人的房产：

编号 父 母 k 孩子 1 ... 孩子 k 房产套数 总面积

其中编号是每个人独有的一个 4 位数的编号；父和母分别是该编号对应的这个人的父母的编号（如果已经过世，则显示-1）； k ($0 \leq k \leq 5$) 是该人的子女的个数；孩子 i 是其子女的编号。

输出格式：

首先在第一行输出家庭个数（所有有亲属关系的人都属于同一个家庭）。随后按下列格式输出每个家庭的信息：

家庭成员的最小编号 家庭人口数 人均房产套数 人均房产面积

其中人均值要求保留小数点后 3 位。家庭信息首先按人均面积降序输出，若有并列，则按成员编号的升序输出。

输入样例：

```
10
6666 5551 5552 1 7777 1 100
1234 5678 9012 1 0002 2 300
8888 -1 -1 0 1 1000
2468 0001 0004 1 2222 1 500
7777 6666 -1 0 2 300
3721 -1 -1 1 2333 2 150
9012 -1 -1 3 1236 1235 1234 1 100
1235 5678 9012 0 1 50
2222 1236 2468 2 6661 6662 1 300
2333 -1 3721 3 6661 6662 6663 1 100
```

输出样例：

```
3
8888 1 1.000 1000.000
0001 15 0.600 100.000
5551 4 0.750 100.000
```

代码长度限制

16 KB

时间限制

400 ms

内存限制

64 MB

栈限制

8192 KB

[2] 实验目的

图的实验旨在帮助学生深入理解图的定义、存储结构、基本操作以及图的各种遍历算法，通过实验操作加深对图论概念的认识，掌握图的构建、遍历和应用，从而提升解决复杂问题的能力，并为后续学习更高级的图算法和应用打下坚实的基础。

[3] 程序清单

实验一：

```
#include <iostream>
#include <queue>
#include <vector>
#include <cstring>
using namespace std;
#define N 100
//邻接表存图，再 BFS 即可
```

```
vector<int> adj[N];
int n, m;
```

```
int BFS(int start) {
    vector<int> dist(n, -1); //起点到各个节点距离
    queue<int> q;
    dist[start] = 0;
    q.push(start);

    while (!q.empty()) {
        int node = q.front(); //处理第一个结点
        q.pop();
        for (int t : adj[node]) {
            if (dist[t] == -1) { //未访问过加一
                dist[t] = dist[node] + 1;
                q.push(t);
            }
        }
    }

    int count = 0;
    for (int i = 0; i < n; i++) {
        if (dist[i] != -1 && dist[i] <= 6) {
            count++;
        }
    }
    return count;
}
```

```
int main() {
    cin >> n >> m;

    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
```

```

        u--; v--; //存下标所以减 1
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    for (int i = 0; i < n; i++)
        printf("%d: %.2f%%\n", i + 1, (double(BFS(i)) / n) * 100);
    return 0;
}

```

```

#include<iostream>
#include<algorithm>
#include<cstdio>
using namespace std;
#define N 10000

struct DATA {
    int id,dad,mum,k,num,area; //k 是子女个数, num 房产数量, area 房产总面积
    int child[10];
};

struct node {
    int id,people; //people 是成员数量
    double num,area; //人均房产数与人均房产面积
    int flag; //标记位, 看是否已经处理
};

int father[N]; //每个节点代表元
int find(int x) { //并查集中找根节点
    while(x != father[x])
        x = father[x];
    return x;
}

void Union(int a,int b) { //并查集, 让 ab 合并, 代表元统一, 成为一个家庭
    int x=find(a);
    int y=find(b);
    if(x>y)    father[x]=y;
    else if(x<y)    father[y]=x;
}

int cmp(node a,node b) {
    if(a.area!=b.area)    return a.area>b.area;
    return a.id<b.id;
}

```

```

int main(){
    struct DATA a[N];
    struct node b[N];

    int visit[N];//辅助访问数组，已经被处理记成 1
    int count=0;//count 为家庭数量

    int n;cin>>n;
    for(int i=0;i<N;i++){//初始化代表元，一开始都是自己
        father[i]=i;
    }

    for(int i=0;i<n;i++){//基础个人信息初始化
        cin>>a[i].id>>a[i].dad>>a[i].mum>>a[i].k;
        visit[a[i].id]=1;
        if(a[i].dad !=-1){
            visit[a[i].dad]=1;
            Union(a[i].dad,a[i].id);
        }
        if(a[i].mum !=-1){
            visit[a[i].mum]=1;
            Union(a[i].mum,a[i].id);
        }
        for(int j=0;j<a[i].k;j++){
            cin>>a[i].child[j];
            visit[a[i].child[j]]=1;
            Union(a[i].child[j],a[i].id);
        }
        cin>>a[i].num>>a[i].area;
    }

    for(int i=0;i<n;i++){//家庭信息初始化
        int id=find(a[i].id);//找到这个家庭的代表元
        b[id].id=id;
        b[id].num+=a[i].num;
        b[id].area+=a[i].area;
        b[id].flag=1;
    }

    for(int i=0;i<N;i++){//统计家庭人员数量与总家庭数
        if(visit[i])    b[find(i)].people++;
        if(b[i].flag)    count++;
    }
}

```

```
for(int i=0;i<N;i++){//统计每个家庭的平均房产数与平均面积
    if(b[i].flag){
        b[i].num=1.0*b[i].num/b[i].people;
        b[i].area=1.0*b[i].area/b[i].people;
    }
}

sort(b,b+N,cmp);
cout<<count<<endl;
for(int i=0;i<count;i++)
    printf("%04d %d %.3f %.3fn",b[i].id,b[i].people,b[i].num,b[i].area);
return 0;
}
```

[4]调试步骤

实验一：

构建图：

实现一个函数来根据输入的边信息构建图的邻接表或邻接矩阵。
确保图的结构正确无误。

实现广度优先搜索（BFS）：

实现 BFS 算法，用于从每个节点出发，找到距离不超过 6 的节点。
确保 BFS 能够正确处理边界情况，如孤立节点或节点间的远距离。

实验二：

构建家庭成员关系图：

实现一个函数来根据输入的家庭成员关系构建图或树结构。
确保图的结构正确无误，每个家庭成员都能正确地链接到其父母和子女。

计算家庭信息：

实现一个函数来计算每个家庭的人口数、人均房产套数和人均房产面积。
确保计算过程中能够正确地累加家庭成员的房产信息，并计算出正确的人均值。

排序输出：

根据计算结果，按照人均房产面积降序排序家庭信息，若人均面积相同，则按成员编号升序排序。
确保排序逻辑正确，输出格式符合要求。

[5] 运行结果:

测试用例

运行结果

编译器输出

运行结果

预期结果

复制内容	复制内容
1 1: 70.00%	1 1: 70.00%
2 2: 80.00%	2 2: 80.00%
3 3: 90.00%	3 3: 90.00%
4 4: 100.00%	4 4: 100.00%
5 5: 100.00%	5 5: 100.00%
6 6: 100.00%	6 6: 100.00%
7 7: 100.00%	7 7: 100.00%
8 8: 90.00%	8 8: 90.00%
9 9: 80.00%	9 9: 80.00%
10 10: 70.00%	10 10: 70.00%

输入样例:

10
6666 5551 5552 1 7777 1 100
1234 5678 9012 1 0002 2 300
8888 -1 -1 0 1 1000
2468 0001 0004 1 2222 1 500
7777 6666 -1 0 2 300
3721 -1 -1 1 2333 2 150
9012 -1 -1 3 1236 1235 1234 1 100
1235 5678 9012 0 1 50
2222 1236 2468 2 6661 6662 1 300
2333 -1 3721 3 6661 6662 6663 1 100

输出样例:

3
8888 1 1.000 1000.000
0001 15 0.600 100.000
5551 4 0.750 100.000

复制内容

1 3
2 8888 1 1.000 1000.000
3 0001 15 0.600 100.000
4 5551 4 0.750 100.000
5

上一次测试于 20 秒前，请注意，测试运行

[6] 分析与思考

对于“六度空间”理论的实验，我们通过图论的视角深入探讨了社交网络的结构特性。这个实验不仅让我们实践了图的构建和遍历算法，尤其是广度优先搜索（BFS），还让我们理解了如何利用图的结构特性来分析和验证社会网络理论。通过 BFS 算法，我们能够找到每个节点在一定距离内可达的节点集合，进而计算出符合“六度空间”理论的节点百分比。这个过程加深了我们对图的连通性、节

点间距离以及图遍历算法在实际应用中重要性的认识。

在“家庭房产”实验中，我们利用图的知识来模拟和分析家庭成员之间的关系，并计算每个家庭的人口数、人均房产套数和人均房产面积。这个实验强化了我们对于图的表示、搜索和处理算法的理解，特别是在处理复杂的关系网络时。通过构建家庭成员关系图，我们能够追踪每个家庭成员的房产信息，并聚合计算整个家庭的统计数据。这个过程不仅提升了对图的深度优先搜索（DFS）和广度优先搜索（BFS）算法的掌握，也让我们认识到图结构在处理现实世界关系数据时的强大能力。

河北工业大学课程共享计划

实验五、查找

河北工业大学课程共享计划

班级：数据 231

学号：235127

姓名：艾洋

[1] 实验内容：

7-1 建立二叉搜索树并查找父结点

分数 300

全屏浏览

切换布局

作者 陈晓梅

单位 广东外语外贸大学

按输入顺序建立二叉搜索树，并搜索某一结点，输出其父结点。

输入格式：

输入有三行：

第一行是 n 值，表示有 n 个结点；

第二行有 n 个整数，分别代表 n 个结点的数据值；

第三行是 x ，表示要搜索值为 x 的结点的父结点。

输出格式：

输出值为 x 的结点的父结点的值。

若值为 x 的结点不存在，则输出：It does not exist.

若值为 x 的结点是根结点，则输出：It doesn't have parent.

输入样例：

2

20

30

20

输出样例：

It doesn't have parent.

####输入样例：

2

20

30

30

输出样例：

20

代码长度限制

16 KB

时间限制

400 ms

内存限制

64 MB

栈限制

8192 KB

7-2 二叉搜索树的删除操作

分数 300

全屏浏览

切换布局

作者 孔德桢

单位 浙大城市学院

给出一棵二叉搜索树（没有相同元素），请输出其删除部分元素之后的层序遍历序列。

删除结点的策略如下：

如果一个结点是叶子结点，则直接删除；

如果一个结点的左子树不为空，则将该结点的值设置为其左子树上各结点中的最大值，并继续删除其左子树上拥有最大值的结点；

如果一个结点的左子树为空但右子树不为空，则将该结点的值设置为其右子树上各结点中的最小值，并继续删除其右子树上拥有最小值的结点。

输入格式：

每个输入文件包含一个测试用例。每个测试用例的第一行包含一个整数 N ($0 < N \leq 100$)，表示二叉搜索树中结点的个数。

第二行给出该二叉搜索树的先序遍历序列，由 N 个整数构成，以一个空格分隔。

第三行给出一个整数 K ($0 < K < N$)，表示待删除的结点个数。最后一行给出 K 个整数，表示待删除的各个结点上的值。必须按输入次序删除结点。题目保证结点一定能被删除。

输出格式：

在一行中输出删除结点后的层序遍历序列。序列中的数字以一个空格分隔，行末不得有多余空格。

输入样例：

```
7
4 2 1 3 6 5 7
2
3 6
```

输出样例：

```
4 2 5 1 7
```

代码长度限制

16 KB

时间限制

400 ms

内存限制

64 MB

栈限制

8192 KB

7-3 代码遗迹

分数 20

全屏浏览

切换布局

作者 刘烨通

单位 河北大学

在一个充满奇妙代码的虚拟世界中，有一位著名的代码探险家 LYET。某天，

他在古老的代码遗迹中发现了一段复杂的代码文档 A，文档中描述了许多函数 B，以及这些函数的详细声明 B'。更令人兴奋的是，LYET 还找到了一段神秘的代码片段 C，据说找到与之相关的函数能解开代码遗迹的终极秘密。然而，探险过程并不容易！LYET 必须确定文档 A 中包含哪些函数 B，而这些函数的声明 B' 中还需要包含这段神秘代码片段 C。面对繁琐的手动筛选，聪明的代码探险家 LYET 决定求助于你这位传奇的代码考古专家。你告诉 LYET：“这太简单了！我们只需要把文档 A 中提到的函数 B，以及声明中包含 C 的函数 B'，找出来匹配一下，就能解决问题！”

为了完成这个任务，你需要编写一个强大的字符串解析器，用来解析文档 A、短代码片段 C，以及函数集合 {B_n} 和它们的声明 {B'_n}。这样，你们就能快速找到哪些函数揭示了代码遗迹的秘密！

形式化地来说，你需要找到那些字符串 B 满足以下条件：

文档 A 中包含 B，同时 B 的函数声明 B' 中包含代码片段 C。

输入格式：

第一行包含一个正整数 n，表示函数的数量，满足 $1 \leq n \leq 104$ 。

第二行包含两个字符串 A 和 C，满足 $1 \leq |A|, |C| \leq 104$ 。

接下来 n 行，每行包含两个字符串 B 和 B'，满足 $1 \leq |B|, |B'| \leq 104$ 。

注意，字符串中仅包含大小写字母。

输出格式：

从小到大，每一行输出一个整数 i，满足：B_i 在文档 A 中，同时函数声明 B'_i 中包含代码片段 C。

输入样例：

```
4
abcdefg z
abcde zoi
abcde oi
abpde zoi
efg bdz
```

输出样例：

```
1
4
```

样例解释：

对于第 1 组数据，B=abcde，B'=zoi，满足 A 中包含 B，且满足 B' 中包含 C。

对于第 4 组数据，B=efg，B'=bdz，满足 A 中包含 B，且满足 B' 中包含 C。

对于第 2 组数据，B=abcde，B'=oi，满足 A 中包含 B，但不满足 B' 中包含 C。

对于第 3 组数据，B=abpde，B'=zoi，不满足 A 中包含 B，满足 B' 中包含 C。

代码长度限制

16 KB

时间限制

1500 ms

内存限制

512 MB

[2] 实验目的

使学生掌握各种查找算法的原理和实现，包括顺序查找、二分查找、哈希查找等，通过实验加深对不同查找技术效率和适用场景的理解，提高解决查找问题的能力，并学会如何根据具体问题选择合适的查找方法以优化性能。

[3] 程序清单

```
实验一：
#include <iostream>
using namespace std;

class TreeNode {
public:
    int data;
    TreeNode *lchild, *rchild;
    TreeNode(int x) : data(x), lchild(nullptr), rchild(nullptr) {}

    TreeNode* insert(TreeNode* root, TreeNode* s) {
        if (root == nullptr)    return s;
        if (s->data < root->data)    root->lchild = insert(root->lchild, s);
        else if (s->data > root->data)    root->rchild = insert(root->rchild, s);
        return root;
    }

    void search(TreeNode* root, int x) {
        if (root == nullptr) {
            cout << "It does not exist." << endl;
            return;
        }

        TreeNode *p = root;
        TreeNode *q = nullptr; //记录 p 的父节点

        while (p) {
            if (x == p->data) {
                if (q == nullptr)    cout << "It doesn't have parent." <<
endl;
                else    cout << q->data << endl;
            }
        }
    }
};
```

```

        return;
    }
    else{
        q = p;
        if (x < p->data)    p = p->lchild;
        else    p = p->rchild;
    }
}
cout << "It does not exist." << endl;
}
};

int main() {
    int n, x; cin >> n >> x;
    TreeNode *root = new TreeNode(x);

    for (int i = 0; i < n - 1; ++i) {
        cin >> x;
        TreeNode *t = new TreeNode(x);
        root = root->insert(root, t);
    }

    cin >> x;
    root->search(root, x);
    return 0;
}

```

```

#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>
using namespace std;

class TreeNode{
public:
    int val;
    TreeNode* left, * right;
    TreeNode(int val) : val(val), left(nullptr), right(nullptr) {}
};

// 根据前序和中序序列重建二叉树
TreeNode* buildTree(vector<int>& pre, vector<int>& in, int preStart, int preEnd,
int inStart, int inEnd) {

```

```

if (preStart > preEnd || inStart > inEnd) return nullptr; // 判空

int rootVal = pre[preStart]; // 前序遍历的第一个元素是根节点
TreeNode* root = new TreeNode(rootVal);

// 在中序遍历中找到根节点的位置
int rootIndexIn = -1;
for (int i = inStart; i <= inEnd; i++) {
    if (in[i] == rootVal) {
        rootIndexIn = i;
        break;
    }
}

// 计算左子树的大小
int leftSize = rootIndexIn - inStart;

// 递归构建左右子树
root->left = buildTree(pre, in, preStart + 1, preStart + leftSize, inStart,
rootIndexIn - 1);
root->right = buildTree(pre, in, preStart + leftSize + 1, preEnd, rootIndexIn
+ 1, inEnd);

return root;
}

TreeNode* findMaxNode(TreeNode* root) {
    while (root && root->right) {
        root = root->right;
    }
    return root;
}

TreeNode* findMinNode(TreeNode* root) {
    while (root && root->left) {
        root = root->left;
    }
    return root;
}

// 删除节点
TreeNode* deleteNode(TreeNode* root, int val) {
    if (!root) return nullptr;

    if (val < root->val) {

```

```

        root->left = deleteNode(root->left, val);
    }
    else if (val > root->val) {
        root->right = deleteNode(root->right, val);
    }
    else {
        if (!root->left && !root->right) {
            // 1. 如果是叶子结点，直接删除
            delete root;
            return nullptr;
        } else if (root->left) {
            // 2. 如果有左子树，将值设置为左子树中的最大值，并删除该
            最大值节点
            TreeNode* maxNode = findMaxNode(root->left);
            root->val = maxNode->val;
            root->left = deleteNode(root->left, maxNode->val); // 删除左子
            树中的最大值
        } else if (root->right) {
            // 3. 如果没有左子树但有右子树，将值设置为右子树中的最小
            值，并删除该最小值节点
            TreeNode* minNode = findMinNode(root->right);
            root->val = minNode->val;
            root->right = deleteNode(root->right, minNode->val); // 删除右
            子树中的最小值
        }
    }
    return root;
}

// 删除指定的多个节点
void deleteNodes(TreeNode* root, const vector<int>& values) {
    for (int val : values) {
        root = deleteNode(root, val);
    }
}

int n,k;
vector<int> outlevelOrder(n-k);
void levelOrder(TreeNode* root) {
    if (root == nullptr) return;

    queue<TreeNode*> q;
    q.push(root);

```



```

while (!q.empty()) {
    TreeNode* current = q.front();
    q.pop();

    //cout << current->val << " ";
    outlevelOrder.push_back(current->val);

    if (current->left != nullptr) {
        q.push(current->left);
    }

    if (current->right != nullptr) {
        q.push(current->right);
    }
}

int main() {
    cin >> n;
    vector<int> pre(n);
    vector<int> in(n);
    for (int i = 0; i < n; i++) {
        cin >> pre[i];
    }
    in = pre;
    sort(in.begin(), in.end());
    TreeNode* root = buildTree(pre, in, 0, n - 1, 0, n - 1);

    cin >> k;
    vector<int> deleteValues(k);
    for (int i = 0; i < k; i++) {
        cin >> deleteValues[i];
    }
    deleteNodes(root, deleteValues);

    levelOrder(root);
    for(int i=0;i<n-k;i++){
        if(i!=n-k-1)    cout<<outlevelOrder[i]<<" ";
        else    cout<<outlevelOrder[i];
    }
    return 0;
}

```

```

#include <iostream>
#include <vector>
#include <string>
using namespace std;

int main() {
    int n;
    string A, C;
    cin >> n;
    cin >> A >> C;
    vector<int> result;

    for (int i = 1; i <= n; ++i) {
        string B1, B2;
        cin >> B1 >> B2;

        bool condition1 = (A.find(B1) != string::npos);
        bool condition2 = (B2.find(C) != string::npos);
        if (condition1 && condition2) {
            result.push_back(i);
        }
    }

    for (int idx : result) {
        cout << idx << endl;
    }
    return 0;
}

```

[4]调试步骤

实验一：
构建二叉搜索树：

实现一个函数来根据输入的数据值构建二叉搜索树。
使用递归或迭代方法都可以，但要确保树的结构正确。

实现查找函数：

实现一个查找函数，用于在二叉搜索树中查找值为 x 的节点，并返回其父节点。

确保在查找过程中能够跟踪当前节点的父节点。

实验二：

构建二叉搜索树：

实现一个函数来根据输入的先序遍历序列构建二叉搜索树。

确保树的结构正确无误。

实现删除操作：

实现一个函数来根据题目中的策略删除指定的节点。

确保在删除节点后，能够正确地更新树的结构，包括找到左子树上的最大值或右子树上的最小值，并进行相应的节点值更新。

层序遍历：

实现一个层序遍历函数，用于在删除操作完成后遍历二叉搜索树。

确保遍历的顺序正确，并能正确地输出遍历序列。

实验三：

实现字符串匹配：

实现一个函数来检查文档 A 是否包含函数 B，以及函数声明 B'是否包含代码片段 C。

使用适当的字符串匹配算法，如 KMP 算法或简单的子串搜索。

匹配和筛选：

实现一个筛选函数，根据匹配结果找出同时满足两个条件的函数 B。

确保筛选逻辑正确，并且能够处理所有输入情况。

[5] 运行结果：

测试用例

运行结果

编译器输出

运行结果

预期结果

复制内容

1 It doesn't have pare

2

复制内容

1 It doesn't have pare

2

输入样例:

7
4 2 1 3 6 5 7
2
3 6

输出样例:

4 2 5 1 7

代码长度限制16 KB

复制内容

1 4·2·5·1·7

输入样例:

4
abcdefg z
abcde zoi
abcde oi
abpde zoi
efg bdz

输出样例:

1
4

运行结果

预期结果

复制内容

1 1

2 4

3

复制内容

1 1

2 4

[6] 分析与思考

针对“建立二叉搜索树并查找父结点”和“二叉搜索树的删除操作”这两个实验，我们深入探讨了二叉搜索树（BST）的结构特性以及与之相关的查找和删除操作。在第一个实验中，我们通过构建 BST 并实现查找父节点的功能，加深了对 BST 中节点关系的理解，这是对基本查找操作的扩展。而在第二个实验中，我们通过删除操作，进一步理解了 BST 的结构变化对查找操作的影响，以及如何维护 BST 的性质。这两个实验都强调了查找知识在处理树结构数据时的重要性，尤其是在进行节点查找和树结构维护时。

对于“代码遗迹”这个实验，我们面临的挑战是如何高效地在大量文本中查找特定的代码片段。这个任务涉及到字符串匹配技术，这是一种特殊的查找操作。我们通过编写字符串解析器来识别和匹配函数声明，这不仅考验了我们对字符串

查找算法的掌握，还要求我们能够有效地处理和比较大量的文本数据。这个实验突出了查找算法在文本处理和信息检索中的应用，尤其是在需要从大量数据中快速定位特定信息时，查找算法的效率和准确性至关重要。

河北工业大学课程共享计划

实验六、排序

班级：数据 231

学号：235127

姓名：艾洋

[1] 实验内容：

7-1 寻找大富翁

分数 300

全屏浏览

切换布局

作者 陈越

单位 浙江大学

胡润研究院的调查显示，截至 2017 年底，中国个人资产超过 1 亿元的高净值人群达 15 万人。假设给出 N 个人的个人资产值，请快速找出资产排前 M 位的大富翁。

输入格式：

输入首先给出两个正整数 N (≤ 106) 和 M (≤ 10)，其中 N 为总人数， M 为需要找出的大富翁数；接下来一行给出 N 个人的个人资产值，以百万元为单位，为不超过长整型范围的整数。数字间以空格分隔。

输出格式：

在一行内按非递增顺序输出资产排前 M 位的大富翁的个人资产值。数字间以空格分隔，但结尾不得有多余空格。

输入样例：

8 3

8 12 7 3 20 9 5 18

输出样例：

20 18 12

代码长度限制

16 KB

时间限制

500 ms

内存限制

64 MB

栈限制

8192 KB

7-2 奥运排行榜

分数 300

全屏浏览

切换布局

作者 陈越

单位 浙江大学

每年奥运会各大媒体都会公布一个排行榜，但是细心的读者发现，不同国家的排行榜略有不同。比如中国金牌总数列第一的时候，中国媒体就公布“金牌榜”；而美国的奖牌总数第一，于是美国媒体就公布“奖牌榜”。如果人口少的国家公布一个“国民人均奖牌榜”，说不定非洲的国家会成为榜魁…… 现在就请你写一个

程序，对每个前来咨询的国家按照对其最有利的方式计算它的排名。

输入格式：

输入的第一行给出两个正整数 N 和 M (≤ 224 ，因为世界上共有 224 个国家和地区)，分别是参与排名的国家和地区的总个数、以及前来咨询的国家的个数。为简单起见，我们把国家从 $0 \sim N-1$ 编号。之后有 N 行输入，第 i 行给出编号为 $i-1$ 的国家的金牌数、奖牌数、国民人口数（单位为百万），数字均为 $[0,1000]$ 区间内的整数，用空格分隔。最后面一行给出 M 个前来咨询的国家的编号，用空格分隔。

输出格式：

在一行里顺序输出前来咨询的国家的排名:计算方式编号。其排名按照对该国家最有利的方式计算；计算方式编号为：金牌榜=1，奖牌榜=2，国民人均金牌榜=3，国民人均奖牌榜=4。输出间以空格分隔，输出结尾不能有多余空格。

若某国在不同排名方式下有相同名次，则输出编号最小的计算方式。

输入样例：

```
4 4
51 100 1000
36 110 300
6 14 32
5 18 40
0 1 2 3
```

输出样例：

```
1:1 1:2 1:3 1:4
```

代码长度限制

16 KB

时间限制

400 ms

内存限制

64 MB

栈限制

8192 KB

7-3 PAT 排名汇总

分数 300

全屏浏览

切换布局

作者 陈越

单位 浙江大学

计算机程序设计能力考试（Programming Ability Test，简称 PAT）旨在通过统一组织的在线考试及自动评测方法客观地评判考生的算法设计与程序设计实现能力，科学的评价计算机程序设计人才，为企业选拔人才提供参考标准（网址 <http://www.patest.cn>）。

每次考试会在若干个不同的考点同时举行，每个考点用局域网，产生本考点的成绩。考试结束后，各个考点的成绩将即刻汇总成一张总的排名表。

现在就请你写一个程序自动归并各个考点的成绩并生成总排名表。

输入格式:

输入的第一行给出一个正整数 N (≤ 100), 代表考点总数。随后给出 N 个考点的成绩, 格式为: 首先一行给出正整数 K (≤ 300), 代表该考点的考生总数; 随后 K 行, 每行给出 1 个考生的信息, 包括考号 (由 13 位整数字组成) 和得分 (为 $[0, 100]$ 区间内的整数), 中间用空格分隔。

输出格式:

首先在第一行里输出考生总数。随后输出汇总的排名表, 每个考生的信息占一行, 顺序为: 考号、最终排名、考点编号、在该考点的排名。其中考点按输入给出的顺序从 1 到 N 编号。考生的输出须按最终排名的非递减顺序输出, 获得相同分数的考生应有相同名次, 并按考号的递增顺序输出。

输入样例:

```
2
5
1234567890001 95
1234567890005 100
1234567890033 95
1234567890002 77
1234567890004 85
4
1234567890013 95
1234567890011 25
1234567890014 100
1234567890042 95
```

输出样例:

```
9
1234567890005 1 1 1
1234567890014 1 2 1
1234567890001 3 1 2
1234567890013 3 2 2
1234567890033 3 1 2
1234567890042 3 2 2
1234567890004 7 1 4
1234567890002 8 1 5
1234567890011 9 2 4
```

鸣谢厦门大学郑旭玲老师补充数据!

代码长度限制

16 KB

时间限制

400 ms

内存限制

64 MB

栈限制

8192 KB

[2] 实验目的

深入理解并掌握各种排序算法的基本原理和实现方法，通过编程实践将理论知识转化为实际操作，比较不同排序算法在不同数据集上的性能，学习如何优化算法以提高效率，并在解决问题的过程中提升编程技能和分析解决问题的能力，最终达到理论与实践相结合的教学效果。

[3] 程序清单

```
#include<iostream>
#include<vector>
using namespace std;

void QuickSort(vector<int>& arr,int low,int high){
    int pivot=arr[low];
    int i=low;
    int j=high;
    if(i>=j)    return;

    while(i<j){
        while(arr[j]>=pivot && i<j)    j--;
        while(arr[i]<=pivot && i<j)    i++;
        if(i<j)    swap(arr[i],arr[j]);
    }

    swap(arr[low],arr[i]);
    QuickSort(arr, low, i - 1);
    QuickSort(arr, i + 1, high);
}

int main(){
    int n,m;cin>>n>>m;
    vector<int> arr(n);
    for(int i=0;i<n;i++){
        cin>>arr[i];
    }
    QuickSort(arr,0,n-1);
    if(n>m){
        for(int i=0;i<m;i++){
            if(i!=m-1)    cout<<arr[n-1-i]<<" ";
            else    cout<<arr[n-1-i];
        }
    }
    else{
```

```

        for(int i=0;i<n;i++){
            if(i!=n-1)    cout<<arr[n-1-i]<<" ";
            else    cout<<arr[n-1-i];
        }
    }
    cout<<endl;
    return 0;
}

```

```

#include<iostream>
#include<vector>
using namespace std;

struct country{
    int gold,medal,people;
    double avgg,avgm;
};

int main(){
    int n,m;cin>>n>>m;
    vector<country> c(n);
    vector<int> q(m);//查询的国家
    vector<int> p(m);//用于判断这个国家在哪种情况下排名最高
    for(int i=0;i<n;i++){
        cin>>c[i].gold>>c[i].medal>>c[i].people;
        c[i].avgg=(double)c[i].gold/c[i].people;
        c[i].avgm=(double)c[i].medal/c[i].people;
    }

    for(int i=0;i<m;i++){
        cin>>q[i];
        for(int k=0;k<4;k++){
            p[k]=0;
        }
        for (int j = 0; j < n; j++) {
            if (c[q[i]].gold >= c[j].gold) p[0]++;
            if (c[q[i]].medal >= c[j].medal) p[1]++;
            if (c[q[i]].avgg >= c[j].avgg) p[2]++;
            if (c[q[i]].avgm >= c[j].avgm) p[3]++;
        }

        int max=0,t=0;
        for(int k=0;k<4;k++){

```

```

        if(max<=p[k]){
            max=p[k];t=k+1;
        }
    }

    if(i!=m-1)    cout<<n+1-max<<":"<<t<<" ";
    else        cout<<n+1-max<<":"<<t;
}
return 0;
}

```

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct People {
    string id;
    int score;
    int frank;//最终排名
    int prank;//考点排名
    int pid;//考点编号
};

bool comparePeople(const People &a, const People &b) {
    if (a.score != b.score)
        return a.score > b.score;
    return a.id < b.id;
}

int main() {
    int n;
    cin >> n;
    vector<vector<People>> points(n);
    vector<People> all_students;//所有人一起的，用来计算最终排名

    for (int i = 0; i < n; i++) {
        int k;cin >> k;
        for (int j = 0; j < k; j++) {
            People p;
            cin >> p.id >> p.score;
            p.pid = i + 1;//考点编号
            points[i].push_back(p);

```

```

    }
}

for (int i = 0; i < n; ++i) { //考点内排名
    sort(points[i].begin(), points[i].end(), comparePeople);

    for (int j = 0; j < points[i].size(); j++) { //排完序了只用检查一下同分的排名，因为是用下标来定的排名
        if (j > 0 && points[i][j].score == points[i][j - 1].score) {
            points[i][j].prank = points[i][j - 1].prank;
        }
        else {
            points[i][j].prank = j + 1;
        }
        all_students.push_back(points[i][j]);
    }
}

sort(all_students.begin(), all_students.end(), comparePeople);

for (int i = 0; i < all_students.size(); ++i) { //最终排名
    if (i > 0 && all_students[i].score == all_students[i - 1].score) {
        all_students[i].frank = all_students[i - 1].frank;
    } else {
        all_students[i].frank = i + 1;
    }
}

cout << all_students.size() << endl;
for (People &student : all_students) {
    cout << student.id << " " << student.frank << " " << student.pid << " "
<< student.prank << endl;
}

return 0;
}

```

[4]调试步骤

实验一：
设计算法：

选择一个合适的算法来解决这个问题。一个简单的方法是使用快速排序或归

并排序对整个数组进行排序，然后输出前 M 个元素。但这种方法在 N 很大时效率不高。

一个更高效的方法是使用“快速选择”算法（Quick Select），它是快速排序的变种，可以部分排序数组，只找到第 k 小的元素，这里的 k 是 M 。

实验二：

对于每个查询国家，我们需要计算该国家最有利的排名。这包括四种计算方式：

金牌榜：按照金牌数降序排序。

奖牌榜：按照奖牌数降序排序。

国民人均金牌榜：按照金牌数与人口的比值降序排序。

国民人均奖牌榜：按照奖牌数与人口的比值降序排序。

每个查询国家的排名应该选择其最有利的的方式。如果多个方式下排名相同，则选择计算方式编号最小的方式。

调试操作：

对于每个计算方式，检查排名是否按照预期排序。

排序时需要注意的是：如果两个国家的排名相同，较小的计算方式编号应当优先。

确保在每个计算方式下都能得到正确的排名，并且在多个查询中选择最有利的排名方式。

实验三：

调试考点内排名计算：

检查每个考点的考生是否按分数正确排序，确保分数相同的考生按考号升序排序。

验证排名计算是否正确，如果多个考生有相同的分数，他们应当具有相同的名次。

调试全局排名计算：

确认全局考生排名的计算正确，特别是对于分数相同的考生，检查名次是否处理得当。

检查全局排名是否考虑了考号的升序，确保在相同分数的情况下，按考号排序。

调试输出格式：

验证输出格式是否符合要求，输出的每行应该包括考号、全局排名、考点编号和考点内排名。

确保全局排名是按非递减顺序输出，并且在相同名次的情况下，考号按升序

排列。

[5] 运行结果:

输出格式:

在一行内按非递增顺序输出资产排名前 M 位的大富翁的个人资产值。
数字间以空格分隔，但结尾不得有多余空格。

输入样例:

```
8 3
8 12 7 3 20 9 5 18
```

输出样例:

```
20 18 12
```

测试用例 1 的输入与输出如下。

若某国在不同排名方式下有相同名次，则输出编号最小的计算方式。

输入样例:

```
4 4
51 100 1000
36 110 300
6 14 32
5 18 40
0 1 2 3
```

输出样例:

```
1:1 1:2 1:3 1:4
```

```
2
5
1234567890001 95
1234567890005 100
1234567890033 95
1234567890002 77
1234567890004 85
4
1234567890013 95
1234567890011 25
1234567890014 100
1234567890042 95
```

输出样例:

```
9
1234567890005 1 1 1
1234567890014 1 2 1
1234567890001 3 1 2
1234567890013 3 2 2
1234567890033 3 1 2
```

测试用例	运行结果	编译器输出
运行结果		
复制内容		复制内容
1	20 18 12	1 20 18 12
2		2

复制内容

```
1 1:1 1:2 1:3 1:4
```

测试用例	运行结果	编译器输出
运行结果		
复制内容		复制内容
1	9	1 9
2	1234567890005 1 1 1	2 1234567890005 1 1 1
3	1234567890014 1 2 1	3 1234567890014 1 2 1
4	1234567890001 3 1 2	4 1234567890001 3 1 2
5	1234567890013 3 2 2	5 1234567890013 3 2 2
6	1234567890033 3 1 2	6 1234567890033 3 1 2
7	1234567890042 3 2 2	7 1234567890042 3 2 2
8	1234567890004 7 1 4	8 1234567890004 7 1 4
9	1234567890002 8 1 5	9 1234567890002 8 1 5
10	1234567890011 9 2 4	10 1234567890011 9 2 4

上一次测试于 12 秒前，请注意，测试运行的任何代码均不作为判分依据

[6] 分析与思考

在这个实验中，我们的目标是快速找出在给定的 N 个人中，资产排名前 M 位的大富翁。这个任务要求我们不仅要理解排序和搜索算法，还要学会如何高效地处理大数据集。调试时，我们需要确保算法能够正确处理各种边界情况，比如极端的输入值和大数据量。此外，我们还需要注意输出格式，确保资产值按非递增顺序输出，并且格式符合题目要求。通过这个实验，我们能够提升对数据结构

和算法的掌握，同时锻炼我们的编程和调试技能。

在“奥运排行榜”实验中，我们面临的挑战是根据每个国家最有利的排名方式来计算其排名。这个任务涉及到多维度数据的处理和比较复杂的逻辑判断。调试过程中，我们需要确保程序能够正确地处理和比较不同国家的金牌数、奖牌数以及国民人均奖牌数。此外，我们还需要关注输出格式，确保排名编号的输出既准确又符合题目要求。通过这个实验，我们能够加深对多条件排序和数据比较算法的理解，同时提高我们的逻辑思维和编程实现能力。

“PAT 排名汇总”实验要求我们将多个考点的成绩合并，并生成一个总排名表。这个任务考验了我们对数据合并和排序算法的应用能力。在调试过程中，我们需要确保程序能够正确地处理每个考点的成绩，并计算出每个考生的总排名。同时，我们还需要特别注意输出格式，确保考生信息的输出顺序和格式符合题目要求。这个实验不仅锻炼了我们处理复杂数据结构的能力，还提高了我们对算法优化和性能调优的认识。通过这个实验，我们能够更好地理解如何在实际应用中实现高效的数据处理和排名生成。

河北工业大学课程共享